

Integration of ThingSpeak with MATLAB and Raspberry Pi

A Comprehensive Guide for Air Quality Monitoring

Faisal Jan*

Zubair Ahmed[†]

Shah Jehan[‡]

April 30, 2026

Contents

1	Introduction	3
2	Software Requirements	3
2.1	Raspberry Pi	3
2.2	Computer	3
3	Installation and Setup	3
3.1	Raspberry Pi Setup	3
3.1.1	Step 1: Install Operating System	3
3.1.2	Step 2: Install Python Packages	3
3.1.3	Step 3: Connect SDS011 Sensor	4
3.2	ThingSpeak Channel Configuration	4
3.2.1	Create a ThingSpeak Account	4
3.2.2	Create a New Channel	4
3.2.3	Configure Fields	4
3.2.4	Retrieve API Keys	4
4	Python Script Implementation	4
4.1	Overview	4
4.2	Script Configuration	5
4.3	Running the Script	5
4.3.1	Interactive Execution	5
4.3.2	Running in Background	6
4.3.3	Auto-start on Boot	6
5	MATLAB Implementation	6
5.1	ThingSpeak Configuration in MATLAB	6
5.2	Data Extraction	6

*<https://github.com/FaisaljanBaloch>

[†]<https://github.com/zoobi1821>

[‡]<https://github.com/shahjehan67>

6	Data Analysis and Visualization	6
6.1	Time-Series Plot	6
6.2	Statistical Analysis	7
6.3	Correlation Analysis	7
6.4	Data Smoothing	7
7	References	8
8	Conclusion	8

1 Introduction

This document provides a comprehensive guide for integrating ThingSpeak cloud platform with MATLAB and Raspberry Pi to create a real-time air quality monitoring system. The system collects data from an SDS011 particulate matter sensor connected to a Raspberry Pi, uploads the data to ThingSpeak, and performs analysis and visualization using MATLAB.

2 Software Requirements

2.1 Raspberry Pi

- Raspberry Pi OS (Raspbian)
- Python 3.7 or higher
- Required Python packages:
 - `sds011lib`: For communicating with SDS011 sensor
 - `requests`: For making HTTP requests to ThingSpeak API

2.2 Computer

- MATLAB R2018b or later
- Internet connection for ThingSpeak access

3 Installation and Setup

3.1 Raspberry Pi Setup

3.1.1 Step 1: Install Operating System

1. Download Raspberry Pi OS from <https://www.raspberrypi.org/downloads/>
2. Use Balena Etcher or similar tool to write the image to SD card
3. Insert SD card and power on the Raspberry Pi

3.1.2 Step 2: Install Python Packages

Connect to Raspberry Pi via SSH and run:

```
pip install sds011lib requests
```

3.1.3 Step 3: Connect SDS011 Sensor

1. Connect the SDS011 sensor to Raspberry Pi via USB-to-TTL converter
2. The sensor will typically be detected as `/dev/ttyUSB0`
3. To verify the connection, run:

```
ls /dev/tty*
```

3.2 ThingSpeak Channel Configuration

3.2.1 Create a ThingSpeak Account

1. Visit <https://thingspeak.com/> and sign up for a free account
2. Verify your email address

3.2.2 Create a New Channel

1. Log in to ThingSpeak
2. Navigate to *Channels* → *My Channels*
3. Click *New Channel*
4. Fill in the following details:
 - **Name:** Air Quality Sensor (or your preferred name)
 - **Description:** Real-time air quality monitoring with PM2.5 and PM10

3.2.3 Configure Fields

Add two fields for your sensor data:

- **Field 1:** PM2.5 (in ug/m3)
- **Field 2:** PM10 (in ug/m3)

3.2.4 Retrieve API Keys

1. Go to the *API Keys* tab of your channel
2. Note your **Channel ID**
3. Copy your **Write API Key** (for Raspberry Pi script)
4. Copy your **Read API Key** (for MATLAB script)

4 Python Script Implementation

4.1 Overview

The Python script on Raspberry Pi reads sensor data and sends it to ThingSpeak at regular intervals.

4.2 Script Configuration

```
from sds011lib import SDS011QueryReader
import time
import requests

# CONFIG
WRITE_API_KEY = "YOUR_WRITE_API_KEY_HERE"
THINGSPEAK_URL = "https://api.thingspeak.com/update"
INTERVAL = 60 # Send data every 60 seconds (minimum 15s for free
               tier)

def read_sds011():
    # Create a query mode reader
    reader = SDS011QueryReader('/dev/ttyUSB0')

    # Query the device
    result = reader.query()

    # Return PM values
    return result.pm25, result.pm10

def send_to_thingspeak(pm25, pm10):
    payload = {
        "api_key": WRITE_API_KEY,
        "field1": pm25,
        "field2": pm10
    }
    response = requests.post(
        "https://api.thingspeak.com/update.json",
        data=payload
    )
    return response.json().get("entry_id", "error")

while True:
    try:
        pm25, pm10 = read_sds011()
        print(f"PM2.5: {pm25} ug/m3 | PM10: {pm10} ug/m3")

        result = send_to_thingspeak(pm25, pm10)
        print(f"Sent to ThingSpeak ? Entry ID: {result}")

    except Exception as e:
        print(f"Error: {e}")

    time.sleep(INTERVAL)
```

4.3 Running the Script

4.3.1 Interactive Execution

```
python air_quality.py
```

4.3.2 Running in Background

```
nohup python air_quality.py &
```

4.3.3 Auto-start on Boot

To automatically run the script on Raspberry Pi startup, add an entry to crontab:

```
crontab -e
```

Add the following line:

```
@reboot python /path/to/air_quality.py &
```

Replace /path/to/ with the actual directory path where the script is located.

5 MATLAB Implementation

5.1 ThingSpeak Configuration in MATLAB

```
channelID = YOUR_CHANNEL_ID;
readAPIKey = 'YOUR_READ_API_KEY';
nobs = 10; % number of observations to retrieve

% Read data from ThingSpeak
data = thingSpeakRead(channelID, 'Fields', [1, 2], ...
    'NumPoints', nobs, 'ReadKey', readAPIKey);
```

5.2 Data Extraction

```
% Extract PM2.5 and PM10 values
pm25 = data(:,1);
pm10 = data(:,2);

% Create time vector
t = 1:nobs;
```

6 Data Analysis and Visualization

6.1 Time-Series Plot

Visualize PM2.5 and PM10 values over time:

```
figure;
plot(t, pm25, '-o', 'LineWidth', 1.5); hold on;
plot(t, pm10, '-s', 'LineWidth', 1.5);
grid on;
xlabel('Sample Number');
ylabel('Measured Value (ug/m3)');
title('Air Quality Over Time');
legend('PM2.5', 'PM10');
```

6.2 Statistical Analysis

```
stats.mean_pm25 = mean(pm25);
stats.mean_pm10 = mean(pm10);
stats.std_pm25 = std(pm25);
stats.std_pm10 = std(pm10);
stats.min_pm25 = min(pm25);
stats.max_pm25 = max(pm25);
stats.min_pm10 = min(pm10);
stats.max_pm10 = max(pm10);

disp('Statistical Summary:');
disp(stats);
```

6.3 Correlation Analysis

```
correlation = corrcoef(pm25, pm10);

figure;
scatter(pm25, pm10, 60, 'filled');
grid on;
xlabel('PM2.5 (ug/m3)');
ylabel('PM10 (ug/m3)');
title('PM2.5 vs PM10 Correlation');
```

6.4 Data Smoothing

Apply moving average to reduce noise:

```
pm25_smooth = movmean(pm25, 3);
pm10_smooth = movmean(pm10, 3);

figure;
plot(t, pm25, 'o--'); hold on;
plot(t, pm25_smooth, 'LineWidth', 2);
plot(t, pm10, 's--');
plot(t, pm10_smooth, 'LineWidth', 2);
grid on;
```

```
legend('PM2.5 Raw', 'PM2.5 Smoothed', 'PM10 Raw', 'PM10 Smoothed'  
);  
title('Smoothed vs Raw Data');
```

7 References

- ThingSpeak Documentation: <https://www.mathworks.com/help/thingspeak/>
- Raspberry Pi Official Guide: <https://www.raspberrypi.org/documentation/>
- SDS011 Sensor Documentation: <https://www.inovafitness.com/en/a/products/95.html>
- MATLAB Data Analysis: <https://www.mathworks.com/help/matlab/>

8 Conclusion

This comprehensive integration of ThingSpeak, Raspberry Pi, and MATLAB provides a robust platform for air quality monitoring and analysis. By following this guide, you can successfully deploy a real-time monitoring system with cloud storage and advanced data analysis capabilities.

For questions or updates regarding this documentation, please refer to the project repository.